In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from zipfile import ZipFile
from sklearn.model_selection import train_test_split
from tensorflow import keras
```

In [2]:
```
with ZipFile("C:\\Users\\Felixeena\\Downloads\\cifar10.zip") as z:
    df = pd.read_csv(z.open("train.csv"),
                     nrows=10000, dtype=np.uint8)

print(df.shape)
print(df.head())
```

```
(10000, 3073)
   pixel_0  pixel_1  pixel_2  pixel_3  pixel_4  pixel_5  pixel_6  pixel_7  \
0       59       43       50       68       98      119      139      145
1      154      126      105      102      125      155      172      180
2      255      253      253      253      253      253      253      253
3       28       37       38       42       44       40       40       24
4      170      168      177      183      181      177      181      184

   pixel_8  pixel_9  ...  pixel_3063  pixel_3064  pixel_3065  pixel_3066  \
0      149      149  ...          58          65          59          46
1      142      111  ...          42          67         101         122
2      253      253  ...          83          80          69          66
3       32       43  ...          39          59          42          44
4      189      189  ...          88          85          82          83

   pixel_3067  pixel_3068  pixel_3069  pixel_3070  pixel_3071  label
0          57         104         140          84          72      6
1         133         136         139         142         144      9
2          72          79          83          83          84      9
3          48          38          28          37          46      4
4          79          78          82          78          80      1

[5 rows x 3073 columns]
```

In [3]:
```
X = df.drop("label", axis=1).to_numpy(dtype=np.float32) / 255.0
y = df["label"].to_numpy()

print(X.shape)
print(y.shape)
```

```
(10000, 3072)
(10000,)
```

In [4]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training:", X_train.shape)
print("Testing:", X_test.shape)
```

```
Training: (8000, 3072)
Testing: (2000, 3072)
```

In [14]:
```
model = keras.Sequential([
    keras.Input(shape=(3072,)),
    keras.layers.Dense(512, activation="relu"),
    keras.layers.Dense(256, activation="relu"),
    keras.layers.Dense(128, activation="relu"),
    keras.layers.Dense(10, activation="softmax")
])
```

In [15]:
```
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
early_stop = keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=3,
    restore_best_weights=True
)
```

In [16]:
```
history = model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.1,
    callbacks=[early_stop]
)
```

```
Epoch 1/10
225/225 ━━━━━━━━━━━━━━━━━━━━ 14s 40ms/step - accuracy: 0.2343 - loss: 2.1008 - val_a
ccuracy: 0.2587 - val_loss: 2.0229
Epoch 2/10
225/225 ━━━━━━━━━━━━━━━━━━━━ 7s 25ms/step - accuracy: 0.3161 - loss: 1.8923 - val_ac
curacy: 0.3187 - val_loss: 1.8962
Epoch 3/10
225/225 ━━━━━━━━━━━━━━━━━━━━ 5s 23ms/step - accuracy: 0.3486 - loss: 1.8049 - val_ac
curacy: 0.3350 - val_loss: 1.8621
Epoch 4/10
225/225 ━━━━━━━━━━━━━━━━━━━━ 12s 31ms/step - accuracy: 0.3638 - loss: 1.7525 - val_a
ccuracy: 0.3500 - val_loss: 1.7880
Epoch 5/10
225/225 ━━━━━━━━━━━━━━━━━━━━ 6s 26ms/step - accuracy: 0.3700 - loss: 1.7148 - val_ac
curacy: 0.3363 - val_loss: 1.8759
Epoch 6/10
225/225 ━━━━━━━━━━━━━━━━━━━━ 6s 25ms/step - accuracy: 0.3907 - loss: 1.6697 - val_ac
curacy: 0.3787 - val_loss: 1.7346
Epoch 7/10
225/225 ━━━━━━━━━━━━━━━━━━━━ 7s 29ms/step - accuracy: 0.4072 - loss: 1.6255 - val_ac
curacy: 0.3638 - val_loss: 1.7835
Epoch 8/10
225/225 ━━━━━━━━━━━━━━━━━━━━ 7s 32ms/step - accuracy: 0.4126 - loss: 1.5999 - val_ac
curacy: 0.3950 - val_loss: 1.7253
Epoch 9/10
225/225 ━━━━━━━━━━━━━━━━━━━━ 8s 34ms/step - accuracy: 0.4367 - loss: 1.5644 - val_ac
curacy: 0.4087 - val_loss: 1.6480
Epoch 10/10
225/225 ━━━━━━━━━━━━━━━━━━━━ 6s 26ms/step - accuracy: 0.4471 - loss: 1.5323 - val_ac
curacy: 0.3988 - val_loss: 1.6621
```

In [17]:
```python
loss, accuracy = model.evaluate(X_test, y_test, verbose=0)

print("Test Loss:", round(loss, 4))
print("Test Accuracy:", round(accuracy * 100, 2), "%")
```

```
Test Loss: 1.6928
Test Accuracy: 38.5 %
```

In [18]:
```python
classes = ["airplane", "automobile", "bird", "cat",
           "deer", "dog", "frog", "horse", "ship", "truck"]

prediction = model.predict(X_test[:1], verbose=0)

predicted_class = np.argmax(prediction[0])

print("Predicted:", classes[predicted_class])
print("Actual:", classes[int(y_test[0])])
```

```
Predicted: horse
Actual: airplane
```

In [19]:
```python
image = X_test[0].reshape(3, 32, 32).transpose(1, 2, 0)

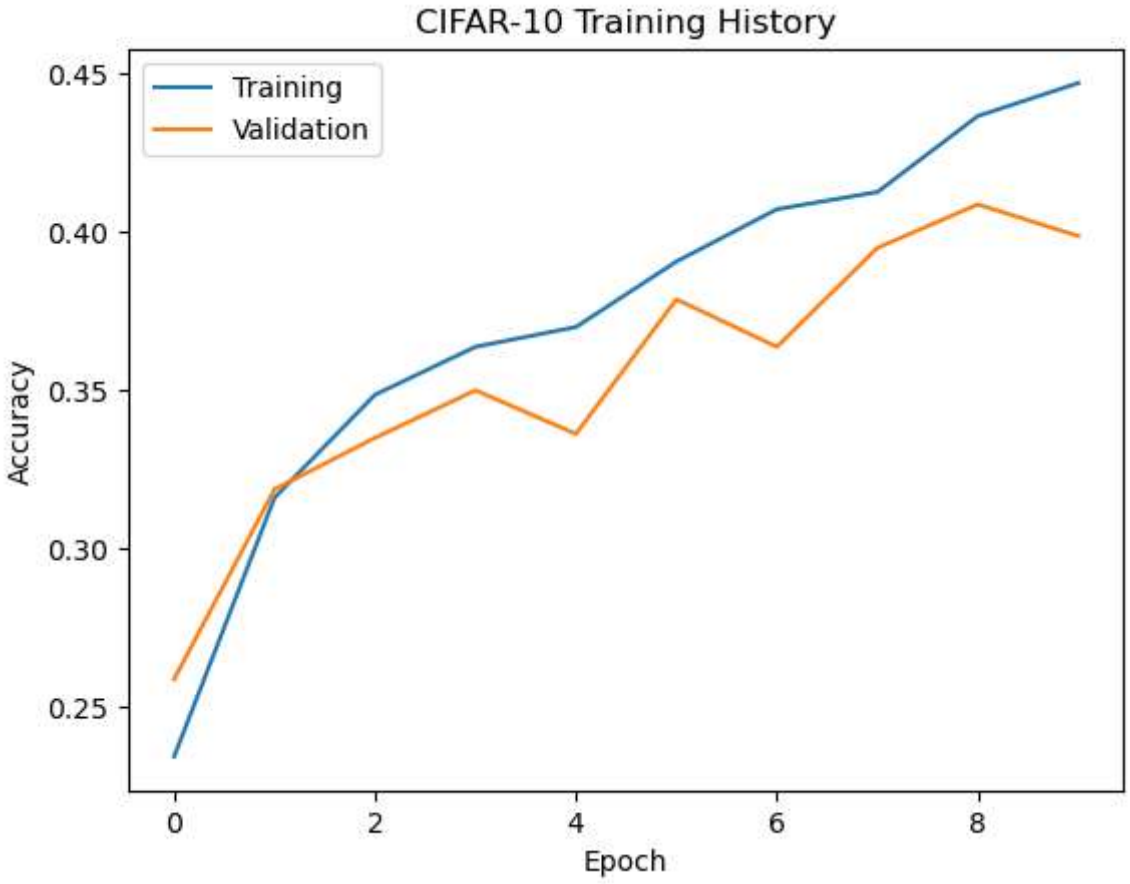
plt.imshow(image)
plt.title(f"Predicted: {classes[predicted_class]} | Actual: {classes[int(y_test[0])
```

```
plt.axis("off")
plt.show()
```



In [20]:
```
plt.plot(history.history["accuracy"], label="Training")
plt.plot(history.history["val_accuracy"], label="Validation")

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("CIFAR-10 Training History")
plt.legend()
plt.show()
```


CIFAR-10 Training History
Training
Validation
Accuracy
Epoch
0.45
0.40
0.35
0.30
0.25
0
2
4
6
8

In [ ]: